



IdentityWallet

VIVA PREPARATION

Viva Questions & Answers

78 questions with clear answers, grouped by topic and marked by difficulty, for the Digital Identity Wallet project.

E · Easy

M · Medium

H · Hard

Student	Cecil Baraik · 2025-BSCIT/004A
Programme	B.Sc. Information Technology
Institution	Amity University Jharkhand

How to use this guide

Viva tips

- Start with the **one-line answer**, then give one example from the project ("for example, when City Cinema checks age...").
- If you don't know something, say what you *do* know and how you would find out — never guess numbers.
- Keep the app running during the viva and **demo**: share "Age check only", open the link, show the green checks, then revoke and verify again.
- Remember the 5 verification checks, the 3 algorithms (ECDSA, AES-GCM, SHA-256) and the 8 passing tests — they come up in many questions.

The 30-second pitch

"IdentityWallet is a digital identity wallet built with Spring Boot and React. Trusted issuers sign credentials with ECDSA; holders keep them encrypted with AES-256-GCM and share only the details a verifier needs — even 'over 18' without revealing their birth date — using salted-hash selective disclosure. Every share is consented, time-limited, view-limited, revocable and logged, and the verifier checks five things automatically. It also has an encrypted vault, two-factor login and a tamper-evident audit log, and all 8 automated security tests pass."

- 1 Project Overview
- 2 Verifiable Credentials and Selective Disclosure
- 3 Cryptography
- 4 Sharing, Consent and Verification
- 5 Authentication and Account Security
- 6 Vault and Data Protection
- 7 Audit Log
- 8 Spring Boot and Backend
- 9 Frontend
- 10 Testing
- 11 Limitations and Future Scope
- 12 Quick-fire Definitions

1 Project Overview

Q1 What is your project? Explain it in two or three lines. Easy

IdentityWallet is a digital identity wallet. Trusted organisations issue **digitally signed credentials** (like a national ID or driving licence) into a person's wallet, and the person can share **only selected details** with a verifier through a QR code or link. The verifier checks authenticity automatically, and the owner sees every access.

Q2 What problem does it solve? Easy

Today we share photocopies of ID cards. They reveal everything (a cinema needs only "over 18" but sees birth date, address and ID number), they are easy to forge, they can't be revoked, and we never know who used them. My project gives **minimal disclosure, tamper-proof verification, consent with expiry/view limits, revocation and an access log**.

Q3 Who are the users of your system? Easy

Four roles: **Wallet holder** (keeps and shares credentials), **Issuer staff** (issues and revokes), **Administrator** (manages the trust registry and users) and **Verifier** (checks a share — needs no account).

Q4 What are the main modules? Easy

Authentication (login, 2FA, logout), Credential management (issue, view, revoke), Sharing & consent, Verification, Encrypted document vault, Administration (trust registry), Audit log and Dashboard.

Q5 What is the technology stack and why? Easy

Backend: Java 17 + Spring Boot 3.3 — strong typing and the built-in Java Cryptography Architecture. **Persistence:** Spring Data JPA/Hibernate with H2 (development) and MySQL (production profile). **Frontend:** React 18 + Vite — fast, component-based, interactive. Libraries: lucide-react icons, qrcode.react for QR, jsQR for camera scanning. **Testing:** JUnit 5 + MockMvc.

Q6 What makes your project different from DigiLocker or Google Drive? Medium

Those store and share **whole files**. My wallet shares **individual facts**, can prove **derived facts** (age over 18) without revealing the source, binds every share to a recipient with the **holder's own signature**, enforces expiry and view limits, and logs every view. The verifier checks cryptographic signatures instead of trusting a file.

2 Verifiable Credentials and Selective Disclosure

Q7 What is a verifiable credential? Easy

A set of claims about a person (name, date of birth...) that is **digitally signed by an issuer**. Anyone with the issuer's public key can verify that it was issued by that issuer and has not been changed. It follows the W3C Verifiable Credentials model of issuer → holder → verifier.

Q8 What is selective disclosure? Easy

Revealing only some claims of a credential while still proving they are genuine. Example: show "Age over 18 = Yes" but hide name, birth date and ID number.

Q9 How did you implement selective disclosure? Hard

Using the **SD-JWT** technique. For each claim the issuer builds a disclosure `[salt, name, value]`, Base64url-encodes it and computes `digest = SHA-256(disclosure)`. Only the list of digests (`_sd`) goes into the signed payload. To share, the holder sends the signed payload plus only the chosen disclosures. The verifier hashes each disclosure and checks the digest is in the signed `_sd` list.

Q10 Why do you add a random salt to every claim? Medium

Without a salt, a verifier could hash every possible value (e.g. all birth dates from 1950 to 2010) and match the digest — a **dictionary/brute-force attack**. A 128-bit random salt makes each digest unpredictable, so hidden values cannot be guessed.

Q11 Can the holder change "age_over_18" from false to true? Medium

No. The changed disclosure produces a different SHA-256 digest which is **not in the signed `_sd` list**, so the "Revealed details" check fails. The holder also cannot add a new digest because that would break the issuer's ECDSA signature.

Q12 What are derived claims? Medium

Claims computed by the issuer at issuance from other data — `age_over_18` and `age_over_21` from the date of birth. They are signed like any other claim, so the holder can prove age **without revealing the date of birth** (data minimisation).

Q13 What happens if the holder's age changes, e.g. they turn 18 after issuance? Hard

`age_over_18` reflects the age at issuance. The issuer re-issues the credential (or a fresh one is issued periodically). In production, credentials with derived age claims would have a shorter validity. This is a known limitation of static derived claims; zero-knowledge range proofs are the advanced alternative.

Q14 What is in the signed payload? Medium

Credential ID, type, title, issuer DID and name, holder DID, issued and expiry dates, the hash algorithm (`sha-256`) and the sorted list of digests `_sd` . **No actual personal values.**

Q15 Why are digests sorted? Hard

So the order of digests does not reveal which claim is which (e.g. the first digest always being the name). Sorting removes that information leak.

3 Cryptography

Q16 Which cryptographic algorithms did you use and where? Easy

ECDSA P-256 with SHA-256 for issuer and holder signatures; **AES-256-GCM** for encrypting vault documents, claims, keys and shares; **SHA-256** for disclosure digests, file integrity, token hashing and the audit chain; **PBKDF2-HMAC-SHA256** for passwords; **HMAC-SHA256** for JWT sessions; **HMAC-SHA1** inside TOTP (as RFC 6238 specifies).

Q17 What is the difference between symmetric and asymmetric encryption? Easy

Symmetric uses one shared key for both encryption and decryption (AES) — fast, used for data. **Asymmetric** uses a key pair: private key to sign/decrypt, public key to verify/encrypt (ECDSA, RSA). I use asymmetric for **signatures** and symmetric for **bulk encryption**.

Q18 What is a digital signature? Easy

A value created with a private key over some data. Anyone with the matching public key can verify it. It proves **authenticity** (who signed) and **integrity** (data not changed), and gives **non-repudiation** (signer can't deny it).

Q19 Why ECDSA P-256 instead of RSA? Medium

ECDSA gives the same security as RSA-3072 with **much smaller keys (256-bit) and signatures (~64-72 bytes)**, and faster signing. It is the algorithm used in ES256 JWTs and many identity standards.

Q20 What is AES-GCM and why GCM mode? Medium

AES is a block cipher; GCM (Galois/Counter Mode) is an **authenticated encryption** mode. It encrypts and also produces a 128-bit **authentication tag**, so any modification of the ciphertext is detected on decryption. Modes like ECB/CBC give confidentiality only.

Q21 What is the IV / nonce in AES-GCM and how do you handle it? Medium

A 12-byte value that must be unique for every encryption with the same key. I generate a **fresh random 12-byte IV** using `SecureRandom` for every encryption and store it in front of the ciphertext (`[IV][ciphertext+tag]`). Reusing an IV with GCM is catastrophic, so it is never reused.

Q22 What is AAD and why did you use it? Hard

Additional Authenticated Data is authenticated but not encrypted. I pass a context string such as `"vault-file:42"` or `"cred:<id>"`. If someone copies one user's encrypted blob into another row, the AAD no longer matches and decryption fails — this stops **ciphertext swapping attacks** by a database attacker.

Q23 What is envelope encryption? Medium

A hierarchy of keys: one **master key** (kept outside the database) encrypts ("wraps") a separate **data key per user**, and the data keys encrypt the actual data. Benefits: a stolen database is useless without the master key, each user's data is isolated, and rotating the master key only requires re-wrapping small data keys.

Q24 Where is the master key stored? Medium

In the environment variable `IDWALLET_MASTER_KEY`, or if not set, in the file `backend/data/master.key` generated on first start. It is **never stored in the database** and is excluded from Git by `.gitignore`. In production it would live in a key management service (AWS KMS, HashiCorp Vault) or an HSM.

Q25 What is hashing? How is it different from encryption? Easy

Hashing is a **one-way** function that gives a fixed-size fingerprint (SHA-256 → 256 bits). It cannot be reversed. Encryption is **two-way** — with the key you get the original back. Hashing is for integrity and passwords; encryption is for confidentiality.

Q26 What properties make SHA-256 suitable? Medium

Pre-image resistance (can't find input from hash), **second pre-image resistance** and **collision resistance** (can't find two inputs with the same hash), plus the **avalanche effect** — changing one bit changes about half the output bits.

Q27 What is a DID in your project? Medium

A Decentralized Identifier: `did:dw:` followed by the first 20 bytes (hex) of `SHA-256(public key)`. Every user and issuer gets one when their key pair is created. It ties an identity to a key, and anyone holding the public key can recompute it.

4 Sharing, Consent and Verification

Q28 What happens when the holder clicks "Create share link"? Medium

The server checks the credential is active, picks the chosen disclosures, generates a random 192-bit token and a nonce, and the holder's private key signs a **holder proof** over (payload hash, disclosures, recipient, nonce, created time, expiry). The presentation is encrypted with a key derived from the token; only the **SHA-256 of the token** is stored. The link `.../v/<token>` and its QR code are returned.

Q29 What is the holder proof and why is it needed? Hard

A signature by the **holder's** private key binding the disclosed claims to a specific recipient and time window. Without it, anyone who intercepted a presentation could **replay** it to another verifier. With it, the verifier knows the real owner consented to share these exact details with them, until that time.

Q30 Why store only the hash of the share token? Hard

Like passwords: if the database leaks, an attacker gets hashes, not working links. The presentation itself is encrypted with a key derived from the token (`SHA-256("idwallet-share-key|" + token)`), so without the original link even the stored data can't be read.

Q31 What checks does the verifier perform? Easy

Five: (1) **Trusted issuer** — issuer DID is registered and not suspended; (2) **Issuer signature** — ECDSA signature over the payload is valid; (3) **Revealed details** — each disclosure's hash is in the signed `_sd`; (4) **Holder consent** — holder's proof signature is valid for this audience; (5) **Time and status** — presentation not expired, credential not revoked or expired. All must pass for "Verified".

Q32 What is the trust registry? Medium

The `issuers` table listing organisations whose public keys the system trusts. Only an admin can add issuers. A signature from an unknown key fails the "Trusted issuer" check, so a hacker can't become an issuer just by generating keys.

Q33 How does revocation work? Easy

The issuer clicks Revoke and gives a reason. The credential's status becomes REVOKED. Every later verification — even of a previously saved JSON copy — fails the "Credential status" check, and the holder can no longer share it.

Q34 How are view limits and expiry enforced? Medium

Each share stores `expiresAt`, `maxViews` and `views`. On every open the server computes the status: revoked → expired → used up → active. Only an active share is decrypted and verified, and `views` is incremented. Every attempt, successful or not, is written to the access log.

Q35 Can a verifier verify offline or later? Medium

The verifier can **save the signed presentation as JSON** and upload it later to re-check. The signatures are checked from the file; the status check needs the server for revocation. Fully offline revocation checking (status lists) is future scope.

5 Authentication and Account Security

Q36 How are passwords stored? Easy

Never in plain text. I use **PBKDF2-HMAC-SHA256** with a 16-byte random salt and **210,000 iterations** (OWASP guidance), stored as `pbkdf2$iterations$salt$hash`. Login recomputes and compares in **constant time**.

Q37 Why not just SHA-256 the password? Medium

SHA-256 is designed to be fast — GPUs compute billions per second, so leaked hashes can be brute-forced quickly. PBKDF2 deliberately repeats the hash 210,000 times (~0.4 s), making brute force about 200,000× slower. The salt prevents rainbow tables and makes identical passwords produce different hashes.

Q38 What is constant-time comparison? Hard

Comparing two byte arrays in a time that doesn't depend on where they first differ (`MessageDigest.isEqual`). A normal comparison stops at the first mismatch, so an attacker measuring response times could learn a secret byte by byte — a **timing attack**.

Q39 Explain your two-factor authentication. Medium

TOTP (RFC 6238). A 160-bit secret is shared with the phone app through a QR code (`otpauth://` URI). Both compute `HMAC-SHA1(secret, floor(time/30))`, truncate it to 6 digits. I accept the current window and one either side for clock drift. The secret is encrypted at rest with the master key.

Q40 How does the login flow work when 2FA is on? Medium

Step 1: correct password returns only an **MFA ticket** — a JWT with `kind = "mfa"` valid for 5 minutes, not a session. Step 2: the ticket plus a correct 6-digit code is exchanged for a real access token. The ticket can't be used to call any other API.

Q41 What is a JWT and how did you secure it? Medium

JSON Web Token: `header.payload.signature`, Base64url. My tokens contain user ID, email, role, kind, issue time, expiry and a random ID, signed with **HMAC-SHA256**. Changing any field breaks the signature. Tokens expire (8 hours by default) and are stored in `sessionStorage` so they vanish when the tab closes.

Q42 How do you protect against brute-force login? Easy

Account lockout — 5 wrong passwords lock the account for 5 minutes; **per-IP rate limiting** — 20 login attempts per 5 minutes, 10 MFA attempts, 5 sign-ups per hour, 60 verifications per minute; and failed attempts are written to the audit log.

Q43 What is account enumeration and how did you prevent it? Hard

Finding out which emails are registered from different error messages or response times. I return the **same message** ("Wrong email or password") in both cases and run a **dummy PBKDF2 hash** for unknown emails, so the response time is the same too.

Q44 How is authorisation enforced? Medium

A Spring `HandlerMethodArgumentResolver` injects an `AuthUser` into any controller method that needs it, and throws 401 if the token is missing or invalid. `AuthUser.require(Role...)` returns 403 for the wrong role. Services also check ownership.

Q45 What is IDOR and how is it handled? Medium

Insecure Direct Object Reference — changing an ID in the URL to access someone else's data. Every service method checks that the record belongs to the logged-in user and returns **404 "not found"** otherwise, so attackers can't even confirm the ID exists.

6 Vault and Data Protection

Q46 How is the document vault secured? Medium

Document number, notes and the file are encrypted with **AES-256-GCM** using the owner's data key, with AAD binding them to the record. The SHA-256 of the original file is stored and checked on every download. Only the title, category and expiry date are plain (for listing and reminders).

Q47 How do you validate uploaded files? Medium

By **magic bytes** — the first bytes of the content (`%PDF-` , PNG `89 50 4E 47` , JPEG `FF D8 FF` , WEBP `RIFF...WEBP`), not the file name or the browser's content-type, which an attacker controls. Max size is 5 MB, and file names are sanitised.

Q48 If the database is stolen, what does the attacker get? Hard

Password hashes (PBKDF2, salted), ciphertext for every private key, data key, TOTP secret, claim and vault file, signed payloads that contain only digests, and hashes of share tokens. Without the master key — stored separately — none of the personal data can be read, and no share link works.

Q49 Which privacy principles does your project follow? Medium

Data minimisation (selective disclosure, derived claims), **purpose limitation** (purpose field), **consent** (holder-signed shares), **storage limitation** (expiry), **transparency** (access log) and **security** (encryption) — matching the DPDP Act 2023 and GDPR principles.

7 Audit Log

Q50 What is a tamper-evident audit log? Medium

Each event stores the hash of the previous event: `hash = SHA-256(prevHash | user | actor | action | detail | ip | time)` . If anyone edits or deletes a row, its hash and every later link stops matching. The admin can click **Verify hash chain** to detect exactly where it broke.

Q51 Is the audit log tamper-proof? Hard

It is **tamper-evident**, not tamper-proof. A DB admin could rewrite the entire chain from a point onward. To prevent that, the latest hash can be periodically published or anchored externally (e.g. in a separate system or a public ledger).

Q52 How do you prevent two events from breaking the chain at the same time? Hard

`AuditService.log` is `synchronized` and runs its own transaction using a `TransactionTemplate` with `REQUIRES_NEW`, so the lock is held until the event is **committed**. No two events can read the same previous hash.

Q53 Why does the audit write use its own transaction? Hard

So events like a failed login are saved even when the main operation is rolled back because it throws an exception. Similarly, `login` uses `noRollbackFor = ApiException.class` so the failed-attempt counter is persisted.

8 Spring Boot and Backend

Q54 Explain the backend architecture. Easy

Layered: **Controllers** (`web`) handle HTTP and validation, **Services** contain business logic and transactions, **Repositories** (Spring Data JPA) access the database, and **Entities** (`domain`) map to tables. `crypto` and `auth` hold security primitives; `config` holds the web configuration and data seeder.

Q55 What is dependency injection? Easy

The framework creates objects (beans) and passes their dependencies in, instead of classes creating them with `new`. I use **constructor injection** — e.g. `CredentialService` receives its repositories and `KeyVault` in the constructor. It makes code loosely coupled and testable.

Q56 What is JPA / Hibernate? Easy

JPA is the Java standard for mapping objects to relational tables; Hibernate is its implementation. Annotations like `@Entity`, `@Id`, `@Column` map classes to tables, and Spring Data generates queries from method names such as `findByHolderIdOrderByIssuedAtDesc`.

Q57 How does SQL injection get prevented? Medium

All queries go through Spring Data JPA, which uses **parameterised (prepared) statements** — user input is never concatenated into SQL. Claim names are additionally validated with a regex.

Q58 What does @Transactional do? Medium

Runs a method inside a database transaction: all changes commit together or roll back together if an exception occurs. `readOnly = true` optimises read-only methods.

Q59 How do you handle errors? Easy

Services throw `ApiException` with an HTTP status. A `@RestControllerAdvice` (`GlobalExceptionHandler`) converts it into `{ "error": "message" }`. Unexpected exceptions are logged on the server and the client only sees "Something went wrong" — no stack traces leak.

Q60 What security headers do you send? Medium

`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (clickjacking), `Referrer-Policy: no-referrer` (share links don't leak in the Referer header), and `Cache-Control: no-store` on API responses so sensitive data isn't cached.

Q61 Why H2 and MySQL both? Easy

H2 is an embedded database — zero setup for development and tests (tests use an in-memory H2). MySQL is for real deployment, enabled with the `mysql` Spring profile.

9 Frontend

Q62 Why React? Easy

Component-based UI, fast updates with the virtual DOM, a huge ecosystem, and it makes interactive screens (share dialog, QR, live checks) simple. Vite gives a fast dev server and build.

Q63 How does the frontend talk to the backend? Easy

Through `fetch` calls in `api.js` to `/api/...`. During development Vite proxies `/api` to `localhost:8080`. The JWT is sent as `Authorization: Bearer <token>`.

Q64 Why sessionStorage and not localStorage for the token? Medium

`sessionStorage` is cleared when the tab closes and is not shared between tabs, reducing the window in which a stolen token can be used. Both are readable by JavaScript, so XSS protection still matters; React escapes output by default.

Q65 How does QR scanning work? Medium

The verifier page opens the camera with `getUserMedia`, draws video frames on a hidden canvas and passes the pixels to `jsQR`, which decodes the QR. The decoded link's token is then verified via the API.

10 Testing

Q66 How did you test the project? Easy

8 automated tests — 5 unit tests for cryptography and 3 integration tests that run the full Spring Boot app with MockMvc — plus manual testing of every page. All 8 pass (`BUILD SUCCESS`).

Q67 Which security scenarios do your tests cover? Medium

Forged claim rejected; payload tampering detected; view limit enforced; revocation detected even on a saved copy; holder can't call issuer APIs (403); user can't read another's credential (404); no token (401); fake PDF upload rejected; MFA wrong code rejected; account locked after 5 wrong passwords (423); audit chain intact.

Q68 How did you verify that your TOTP implementation is correct? Medium

Against the official **RFC 6238 test vector**: secret "12345678901234567890" at time 59 seconds must give 94287082 (last 6 digits **287082**). My test checks exactly that.

11 Limitations and Future Scope

Q69 What are the limitations of your project? Medium

Keys are **custodial** (encrypted on the server, not on the user's phone); the format is SD-JWT-style, not fully standard, so no interoperability yet; revocation needs a live server check; rate limits are in memory (single server); derived age claims are static.

Q70 What would you add in future? Easy

A mobile wallet with keys in the secure enclave and biometrics; passkeys (WebAuthn); standard SD-JWT VC / W3C VC 2.0 and OpenID4VP; bitstring status lists for revocation; zero-knowledge proofs (BBS+) for unlinkable presentations; recovery codes; Docker deployment with HTTPS.

Q71 Why didn't you use blockchain? Medium

Integrity comes from **digital signatures** and a trust registry; blockchain would add cost and complexity without adding much, and personal data must stay off-chain anyway for privacy laws. (My other project, CredentialChain, explores the blockchain approach.)

Q72 How would you deploy it in production? Medium

Build the React app into the Spring Boot jar (`npm run build:jar` , `mvnw package`), run it behind Nginx with HTTPS, use MySQL, inject the master key and JWT secret from a secrets manager, add Redis for rate limiting and run in Docker.

12 Quick-fire Definitions

Q73 Authentication vs authorisation? Easy

Authentication = who are you (login, 2FA). **Authorisation** = what are you allowed to do (roles, ownership).

Q74 Integrity, confidentiality, availability (CIA)? Easy

Confidentiality — only authorised people read data (AES encryption). Integrity — data isn't altered (signatures, hashes, GCM). Availability — the system stays usable (rate limits, lockout duration instead of permanent lock).

Q75 Non-repudiation? Easy

The signer cannot deny signing. Issuer signatures and holder proofs provide it.

Q76 Salt vs nonce vs IV? Medium

Salt — random value added before hashing so equal inputs give different hashes. **Nonce** — "number used once", e.g. in the holder proof to make each proof unique. **IV** — the nonce of a cipher mode like AES-GCM.

Q77 Base64 vs Base64url? Easy

Base64url replaces `+` and `/` with `-` and `_` and drops `=` padding, so the result is safe in URLs and file names — used for share tokens, signatures and disclosures.

Q78 What is the OWASP Top 10? Easy

A list of the ten most critical web application security risks. My project addresses Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Authentication Failures, Integrity Failures and Logging Failures.